

Student Name: Jinpeng Zhu
Student ID: 1907843
Date: 2026-01-27

Intro to Finite Element Method (CIV E – 665)
Assignment No. 3



1 Question 1

Given equations:

$$EI \frac{d^4 w}{dx^4} = q(x) \quad \text{for } 0 \leq x \leq L \quad (1)$$

Where: $q(x) = -950x - 50$, $L = 1 \text{ m}$ and $EI = 10^6 \text{ Nm}^2$

$$EI \int_0^L \frac{d^2 v}{dx^2} \frac{d^2 w}{dx^2} dx - \left[\frac{dv}{dx} EI \frac{d^2 w}{dx^2} \right]_0^L + \left[v EI \frac{d^3 w}{dx^3} \right]_0^L = \int_0^L v q dx \quad (2)$$

$$W(x) = c_2 x^3 + c_1 x^2 \quad (3)$$

$$v(x) = x^3 + x^2 \quad (4)$$

$$\theta_1 = x^2 \quad \text{and} \quad \theta_2 = x^3 \quad (5)$$

1.1 i

For this cantilever beam problem, we at least need to satisfy two essential boundary conditions at the fixed end ($x=0$) to ensure the displacement and slope are both zero.

So, we can get the essential boundary conditions at the fixed: (1) at $x = 0$, $W(0) = 0$; (2) at $x = 0$, $W'(0) = 0$

Substitute $x=0$ into $W(x)$

$$W(0) = c_2 \times 0 + c_1 \times 0 = 0$$

So, the essential boundary condition (1) is satisfied.

For essential boundary condition 2, the first derivative of $w(x)$:

$$W'(x) = \frac{dW}{dx} = 3c_2 x^2 + 2c_1 x \quad (6)$$

Substitute $x=0$ into $W'(x)$

$$W'(0) = \frac{dW}{dx} = 3c_2(0)^2 + 2c_1(0) = 0$$

Therefore, the essential boundary condition 2 is also satisfied.

1.2 ii

Based on solution (i), at $x = 0$, we have:

$$\begin{aligned}\left.\frac{d^2w}{dx}\right|_{x=0} &= 0 \\ \left.\frac{d^3w}{dx}\right|_{x=0} &= 0 \\ \left.\frac{d^2v}{dx}\right|_{x=0} &= 0 \\ \left.\frac{d^3v}{dx}\right|_{x=0} &= 0\end{aligned}\tag{7}$$

applying the essential boundary condition, the Equation 2 can be simplified as:

$$EI \left[\left. \frac{dv}{dx} \frac{d^2w}{dx} \right|_0^L - \int_0^L \frac{dv}{dx} \frac{d^3w}{dx} dx \right] - \left(\left. \frac{dv}{dx} EI \frac{d^2w}{dx^2} \right|_0^L + \left(v EI \frac{d^3w}{dx^3} \right) \right|_0^L = \int_0^L v q dx\tag{8}$$

Using integration by parts on the term $\int_0^L \frac{dv}{dx} \frac{d^3w}{dx} dx$ in Equation 8,

$$\int_0^L \frac{dv}{dx} \frac{d^3w}{dx} dx = \left(v \frac{d^3w}{dx^3} \right) \Big|_0^L - \int_0^L \frac{d^2w}{dx^2} \frac{d^2v}{dx^2} dx\tag{9}$$

Substitute Equation 9 to Equation 8, we can get:

$$EI \int_0^L \frac{d^2w}{dx^2} \frac{d^2v}{dx^2} dx = \int_0^L v q dx\tag{10}$$

And then, Building stiffness matrix

$$K_{ij} = EI \int_0^L \frac{d^2v_i}{dx^2} \frac{d^2v_j}{dx^2} dx\tag{11}$$

$$F_i = \int_0^L v_i q(x) dx\tag{12}$$

Given $\theta_1 = x^2$ and $\theta_2 = x^3$,

$$\begin{aligned}\frac{d^2\theta_1}{dx^2} &= 2 \\ \frac{d^2\theta_2}{dx^2} &= 6x\end{aligned}$$

For each element of the stiffness matrix:

$$\begin{aligned}
K_{11} &= EI \int_0^L (2)(2)dx = 4x \Big|_0^{L=1} = 4EI \\
K_{12} &= EI \int_0^L (2)(6x)dx = 6x^2 \Big|_0^{L=1} = 6EI \\
K_{21} &= EI \int_0^L (6x)(2)dx = 6x^2 \Big|_0^{L=1} = 6EI \\
K_{22} &= EI \int_0^L (6x)(6x)dx = 12x^3 \Big|_0^{L=1} = 12EI
\end{aligned}$$

so,

$$[K] = EI \begin{bmatrix} 4 & 6 \\ 6 & 12 \end{bmatrix}$$

For $F(x)$,

$$\begin{aligned}
F_1 &= \int_0^1 x^2(-950x - 50)dx = \int_0^1 (-950x^3 - 50x^2)dx \\
&= \left[\frac{-950x^4}{4} - \frac{50x^3}{3} \right]_0^1 = -\frac{1525}{6} \\
F_2 &= \int_0^1 x^3(-950x - 50)dx = \int_0^1 (-950x^4 - 50x^3)dx \\
&= \left[\frac{-950x^5}{5} - \frac{50x^4}{4} \right]_0^1 = -\frac{405}{2}
\end{aligned}$$

Therefore, the system is:

$$10^6 \begin{bmatrix} 4 & 6 \\ 6 & 12 \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} = \begin{bmatrix} -\frac{1525}{6} \\ -\frac{405}{2} \end{bmatrix} \quad (13)$$

Solve the system, $c_1 = -1.52916667 \times 10^{-4}$ and $c_2 = 5.95833333 \times 10^{-5}$

1.3 iii

Matplotlib is employed to visualize and compare the exact solution $w(x)$ and the approximate solution $W(x)$ derived from the finite element method. the source code is shown Section 3.1.

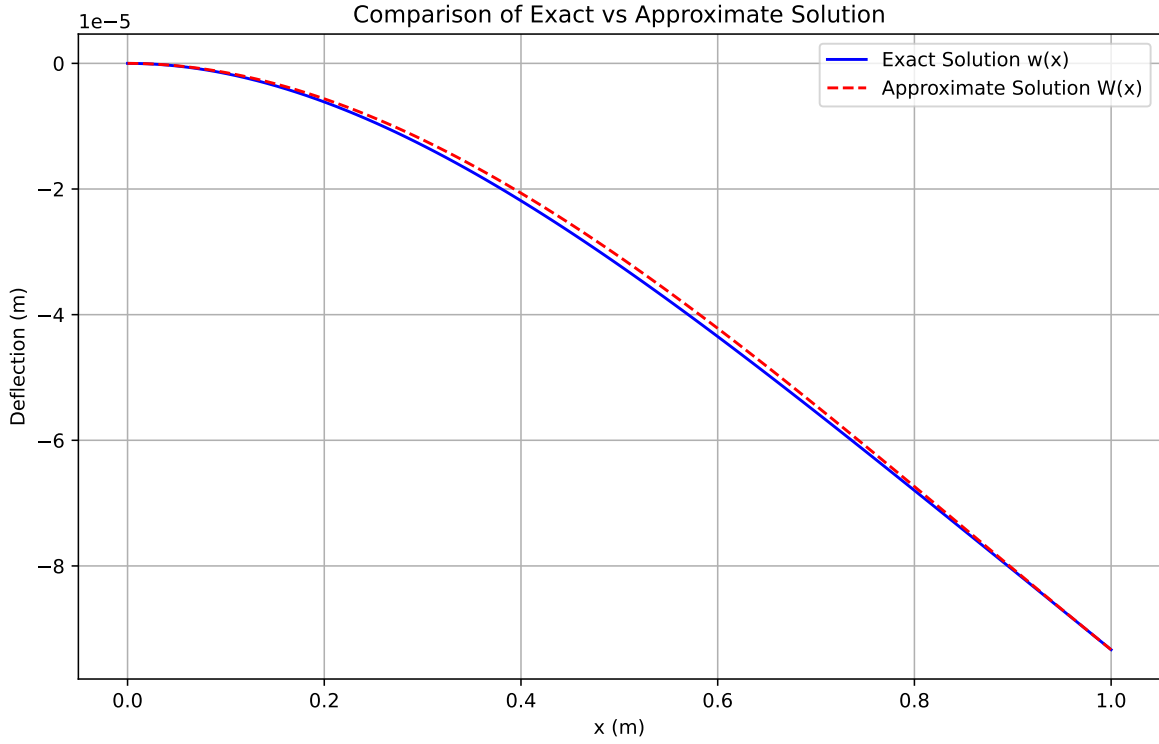


Figure 1: Comparison of Exact vs Approximate Solution

The plot demonstrates that the approximate solution $W(x)$ (red dashed line) is in excellent agreement with the exact solution $w(x)$ (blue solid line).

Both solutions satisfy the essential boundary conditions at the fixed support ($x = 0$), where the deflection is strictly zero. Despite $W(x)$ being a lower-order polynomial (cubic) compared to the exact solution (quintic), it captures the maximum deflection at the free end ($x = 1$) with high accuracy.

2 Question 2

Since their diameters are different, we need to divide the stepped bar into three elements:

1. Element 1 (steel): from $x = 0$ to $x = 16$ inches, $A_1 = 8 \text{ in}^2$, $E_1 = 30 \times 10^6 \text{ psi}$
2. Element 2 (aluminum): from $x = 16$ inches to $x = 32$ inches, $A_2 = 6 \text{ in}^2$, $E_2 = 10 \times 10^6 \text{ psi}$
3. Element 3 (brass): from $x = 32$ inches to $x = 48$ inches, $A_3 = 4 \text{ in}^2$, $E_3 = 15 \times 10^6 \text{ psi}$

Correspondingly, we have four nodes:

1. Node 1: at $x = 0$ inches
2. Node 2: at $x = 16$ inches
3. Node 3: at $x = 32$ inches
4. Node 4: at $x = 48$ inches

2.1 I

Since these three elements have same length, so for each linear 1D bar element, the stiffness matrix $[k^e]$ is given by:

$$k^e = \frac{EA}{L} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

For element 1 (Steel, Nodes 1-2):

$$k^{(1)} = \frac{(30 \times 10^6)(8)}{16} = 15 \times 10^6 \text{ lb/in}$$

$$[k^{(1)}] = 10^6 \begin{bmatrix} 15 & -15 \\ -15 & 15 \end{bmatrix}$$

For element 2 (Aluminum, Nodes 2-3):

$$k^{(2)} = \frac{(10 \times 10^6)(6)}{16} = 3.75 \times 10^6 \text{ lb/in}$$

$$[k^{(2)}] = 10^6 \begin{bmatrix} 3.75 & -3.75 \\ -3.75 & 3.75 \end{bmatrix}$$

For element 3 (Brass, Nodes 3-4):

$$k^{(3)} = \frac{(15 \times 10^6)(4)}{16} = 3.75 \times 10^6 \text{ lb/in}$$

$$[k^{(3)}] = 10^6 \begin{bmatrix} 3.75 & -3.75 \\ -3.75 & 3.75 \end{bmatrix}$$

Element Force Vectors:

The problem states there are no body forces ($b(x) = 0$). Therefore, the distributed force vector $\{f\}$ for every element is zero.

$$\{f^{(1)}\} = \{f^{(2)}\} = \{f^{(3)}\} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

2.2 II

We assemble the local matrices into a global 4×4 stiffness matrix $[K]$.

Global Stiffness Matrix $[K]$:

$$[K] = 10^6 \begin{bmatrix} k_{11}^{(1)} & k_{12}^{(1)} & 0 & 0 \\ k_{21}^{(1)} & k_{22}^{(1)} + k_{11}^{(2)} & k_{12}^{(2)} & 0 \\ 0 & k_{21}^{(2)} & k_{22}^{(2)} + k_{11}^{(3)} & k_{12}^{(3)} \\ 0 & 0 & k_{21}^{(3)} & k_{22}^{(3)} \end{bmatrix} \quad (14)$$

Substituting values:

- $K_{22} = 15 + 3.75 = 18.75$
- $K_{33} = 3.75 + 3.75 = 7.5$

Global Force Vector $\{F\}$:

We assign external point loads to the corresponding nodes.

- **Node 1:** Reaction force F_1
- **Node 2:** Two 2500 lb loads pointing right = +5000 lb
- **Node 3:** No external load = 0
- **Node 4:** One 2000 lb load pointing left = -2000 lb

$$\{F\} = \begin{bmatrix} F_1 \\ 5000 \\ 0 \\ -2000 \end{bmatrix}$$

System Equation:

$$10^6 \begin{bmatrix} 15 & -15 & 0 & 0 \\ -15 & 18.75 & -3.75 & 0 \\ 0 & -3.75 & 7.5 & -3.75 \\ 0 & 0 & -3.75 & 3.75 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} = \begin{bmatrix} F_1 \\ 5000 \\ 0 \\ -2000 \end{bmatrix}$$

2.3 III

Boundary Condition: The bar is fixed at the wall (Node 1), so displacement is zero.

$$u_1 = 0$$

We can remove the first row and first column (the “reduced” system) to solve for the unknown displacements u_2, u_3, u_4 .

$$10^6 \begin{bmatrix} 18.75 & -3.75 & 0 \\ -3.75 & 7.5 & -3.75 \\ 0 & -3.75 & 3.75 \end{bmatrix} \begin{bmatrix} u_2 \\ u_3 \\ u_4 \end{bmatrix} = \begin{bmatrix} 5000 \\ 0 \\ -2000 \end{bmatrix}$$

Use Matlab solving (the code please see Section 3.2):

$$u_2 = 2.000000 \times 10^{-04} \quad u_3 = -3.333333 \times 10^{-04} \quad u_4 = -8.666667 \times 10^{-04}$$

Therefore, the $u(x)$ for each element is:

$$\begin{aligned} u^1(x) &= \frac{u_2 - u_1}{16}x + u_1 = 1.25 \times 10^{-5}x \quad \text{for } 0 \leq x \leq 16 \\ u^2(x) &= \frac{u_3 - u_2}{16}(x - 16) + u_2 = -3.333333 \times 10^{-5}x + 8.666667 \times 10^{-4} \quad \text{for } 16 \leq x \leq 32 \\ u^3(x) &= \frac{u_4 - u_3}{16}(x - 32) + u_3 = -3.333333 \times 10^{-5}x - 3.333333 \times 10^{-4} \quad \text{for } 32 \leq x \leq 48 \end{aligned}$$

2.4 IV

The displacement, strain, and stress as a function of the axial position x is shown as Figure 2. The plot code please see Section 3.

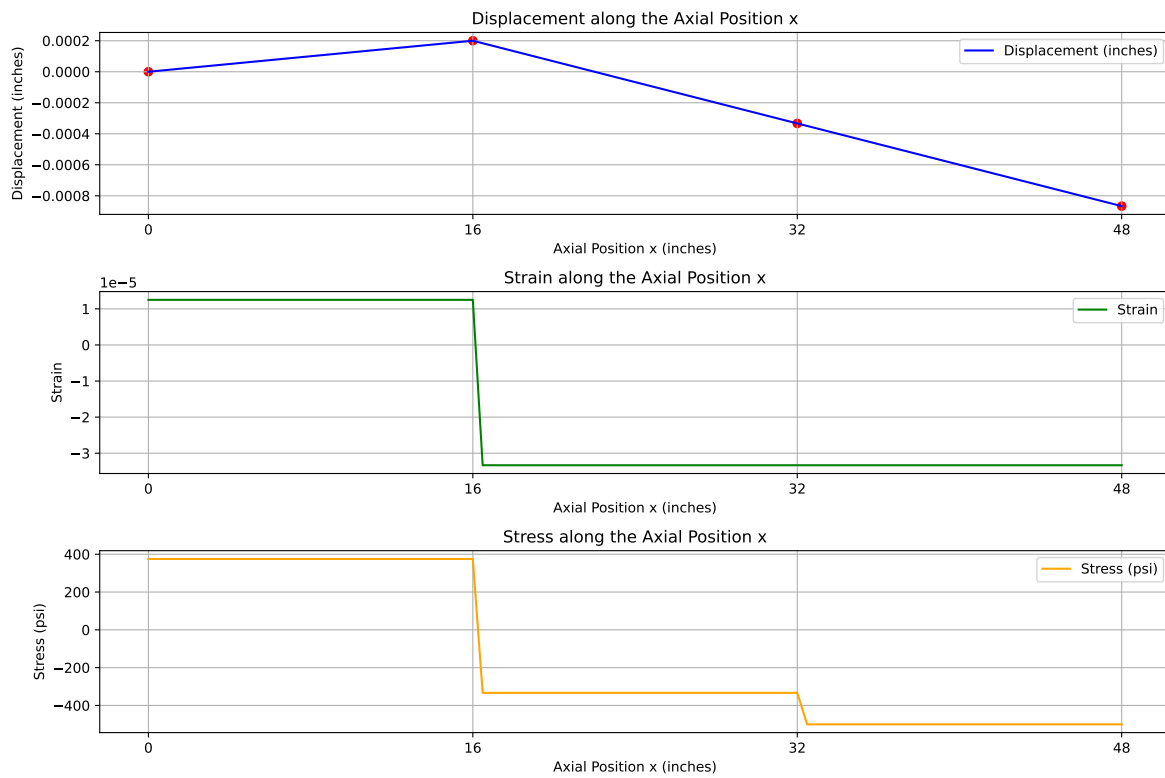


Figure 2: Displacement, strain, and stress along the axial position x

3 Appendix

3.1 The Python code for Question 1 ii

```
#| label: fig-exact-vs-approx
#| caption: Comparison of Exact vs Approximate Solution
#| echo: false
import numpy as np
import matplotlib.pyplot as plt

# Parameters
L = 1
EI = 1e6

# Stiffness Matrix K and Force Vector F
K = EI * np.array([[4, 6], [6, 12]])
F = np.array([-1525/6, -405/2])

# Solve for coefficients c1 and c2
C = np.linalg.solve(K, F)
print( C )
c1, c2 = C
```

```

# Define x range
x = np.linspace(0, L, 100)

# Exact solution w(x)
w_exact = (1/EI) * (-950/120 * x**5 - 50/24 * x**4 + 525/6 * x**3 - 1025/6 * x**2)

# Approximate solution W(x)
W_approx = c1 * x**2 + c2 * x**3

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(x, w_exact, label='Exact Solution w(x)', linestyle='-', color='blue')
plt.plot(x, W_approx, label='Approximate Solution W(x)', linestyle='--', color='red')
plt.title('Comparison of Exact vs Approximate Solution')
plt.xlabel('x (m)')
plt.ylabel('Deflection (m)')
plt.legend()
plt.grid(True)
plt.show()

```

3.2 The Matlab code for Question 2 III

```

% Define the coefficient matrix
A_base = [18.75, 3.75, 0;
          3.75, 7.5, 3.75;
          0, 3.75, 3.75];
A = 10^6 * A_base;

% Define the constants vector
b = [5000; 0; 2000];

% Solve for u using the backslash operator (Gaussian elimination)
u = A \ b;

% Display the results
fprintf('u2 = %e\n', u(1));
fprintf('u3 = %e\n', u(2));
fprintf('u4 = %e\n', u(3));

```

3.3 The Python code for Question 2 IV

```

#| label: fig-displacement
#| caption: Displacement, strain, and stress along the axial position x

import numpy as np
import matplotlib.pyplot as plt

# Node positions
x_nodes = [0, 16, 32, 48] # in inches
# Displacements at nodes
u_nodes = [0, 2.0e-4, -3.333333e-4, -8.666667e-4] # in inches

```

```

# Create finer x values for plotting
x_fine = np.linspace(0, 48, 100)
# Interpolate displacements
u_fine = np.piecewise(x_fine, [x_fine <= 16, (x_fine > 16) & (x_fine <= 32), x_fine > 32],
                      [lambda x: (u_nodes[1]-u_nodes[0])/(x_nodes[1]-x_nodes[0])*(x - x_nodes[0]) + u_nodes[0],
                        lambda x: (u_nodes[2]-u_nodes[1])/(x_nodes[2]-x_nodes[1])*(x - x_nodes[1]) + u_nodes[1],
                        lambda x: (u_nodes[3]-u_nodes[2])/(x_nodes[3]-x_nodes[2])*(x - x_nodes[2]) + u_nodes[2]])
plt.figure(figsize=(12, 8))
# Displacement plot
plt.subplot(3, 1, 1)
plt.plot(x_fine, u_fine, label='Displacement (inches)', color='blue')
plt.scatter(x_nodes, u_nodes, color='red') # Mark nodes
plt.title('Displacement along the Axial Position x')
plt.xlabel('Axial Position x (inches)')
plt.ylabel('Displacement (inches)')
plt.xticks(x_nodes)
plt.grid()
plt.legend()
# Strain plot
plt.subplot(3, 1, 2)
strain_fine = np.piecewise(x_fine, [x_fine <= 16, (x_fine > 16) & (x_fine <= 32), x_fine > 32],
                           [lambda x: (u_nodes[1]-u_nodes[0])/(x_nodes[1]-x_nodes[0]),
                             lambda x: (u_nodes[2]-u_nodes[1])/(x_nodes[2]-x_nodes[1]),
                             lambda x: (u_nodes[3]-u_nodes[2])/(x_nodes[3]-x_nodes[2])])
plt.plot(x_fine, strain_fine, label='Strain', color='green')
plt.title('Strain along the Axial Position x')
plt.xticks(x_nodes)
plt.xlabel('Axial Position x (inches)')
plt.ylabel('Strain')
plt.grid()
plt.legend()
# Stress plot
plt.subplot(3, 1, 3)
stress_fine = np.piecewise(x_fine, [x_fine <= 16, (x_fine > 16) & (x_fine <= 32), x_fine > 32],
                           [lambda x: 30e6 * (u_nodes[1]-u_nodes[0])/(x_nodes[1]-x_nodes[0]),
                             lambda x: 10e6 * (u_nodes[2]-u_nodes[1])/(x_nodes[2]-x_nodes[1]),
                             lambda x: 15e6 * (u_nodes[3]-u_nodes[2])/(x_nodes[3]-x_nodes[2])])
plt.plot(x_fine, stress_fine, label='Stress (psi)', color='orange')
plt.title('Stress along the Axial Position x')
plt.xlabel('Axial Position x (inches)')
plt.xticks(x_nodes)
plt.ylabel('Stress (psi)')
plt.grid()
plt.legend()
plt.tight_layout()
plt.show()

```